

```
# Data handling libraries
import pandas as pd
import numpy as np

# Visualization libraries
import matplotlib.pyplot as plt
import seaborn as sns

# PyTorch for deep learning
import torch

# Hugging Face libraries
from datasets import load_dataset
from transformers import (
    AutoTokenizer,
    AutoModelForSequenceClassification,
    TrainingArguments,
    Trainer
)

# Evaluation metrics
from sklearn.metrics import accuracy_score, precision_recall_fscore_support, confusion_matrix, cla
```

```
# Load IMDb dataset
dataset = load_dataset("imdb")

print(dataset)
```

```
/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:94: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (https://huggingface.co/settings/tokens)
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or datasets.
warnings.warn(

README.md:      7.81k/? [00:00<00:00, 582kB/s]

plain_text/train-00000-of-100000.parquet: 21.0M/21.0M [00:00<00:00, 32.9MB/s]
00001.parquet: 100%

plain_text/test-00000-of-100000.parquet: 20.5M/20.5M [00:00<00:00, 31.3MB/s]
00001.parquet: 100%

plain_text/unsupervised-00000-of-100000.p(...): 42.0M/42.0M [00:00<00:00, 22.2MB/s]
00001.p(...): 100%

Generating train split: 100%                25000/25000 [00:00<00:00, 91157.24 examples/s]
Generating test split: 100%                 25000/25000 [00:00<00:00, 108056.11 examples/s]
Generating unsupervised split: 100%         50000/50000 [00:00<00:00, 135016.36 examples/s]
DatasetDict({
  train: Dataset({
    features: ['text', 'label'],
    num_rows: 25000
  })
  test: Dataset({
    features: ['text', 'label'],
    num_rows: 25000
  })
  unsupervised: Dataset({
    features: ['text', 'label'],
    num_rows: 50000
  })
})
```

```
print(dataset["train"][0])
```

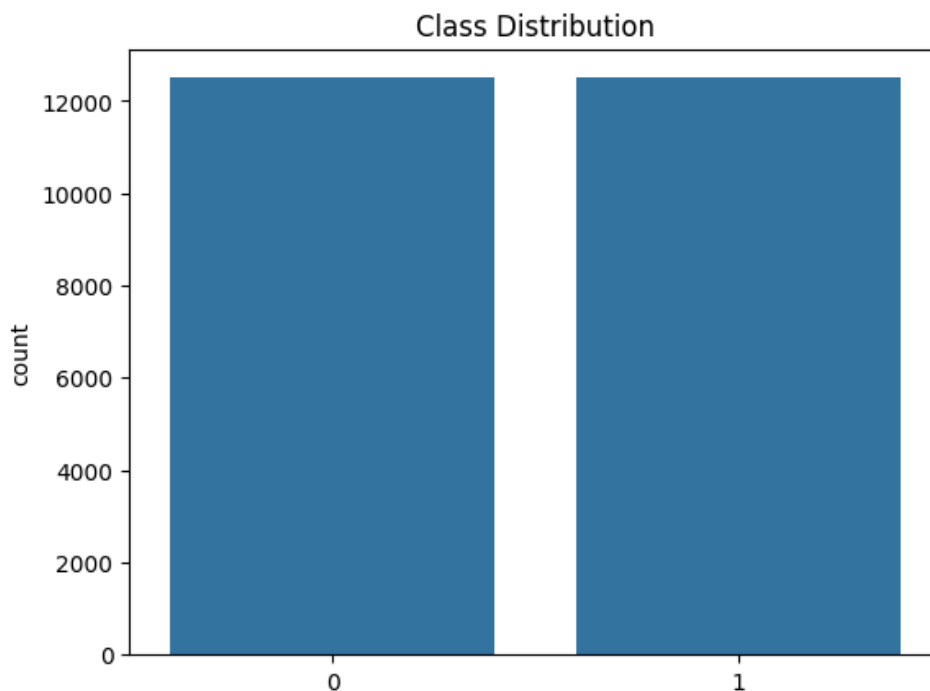
```
{'text': 'I rented I AM CURIOUS-YELLOW from my video store because of all the controversy that sur
```

```
train_labels = dataset["train"]["label"]
```

```
sns.countplot(x=train_labels)
plt.title("Class Distribution")
plt.show()
```

```
print("Class 0 (Negative):", train_labels.count(0))
```

```
print("Class 1 (Positive):", train_labels.count(1))
```



```
Class 0 (Negative): 12500
```

```
Class 1 (Positive): 12500
```

```
tokenizer = AutoTokenizer.from_pretrained("bert-base-uncased")
```

```
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable f
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated requests to the HF Hu
```

```
config.json: 100%
```

```
570/570 [00:00<00:00, 8.93kB/s]
```

```
tokenizer_config.json: 100%
```

```
48.0/48.0 [00:00<00:00, 736B/s]
```

```
vocab.txt: 100%
```

```
232k/232k [00:00<00:00, 2.75MB/s]
```

```
tokenizer.json: 100%
```

```
466k/466k [00:00<00:00, 4.42MB/s]
```

```
def tokenize_function(example):
    return tokenizer(
        example["text"],
        padding="max_length",
        truncation=True,
        max_length=256
    )
```

```
tokenized_datasets = dataset.map(tokenize_function, batched=True)
```

```

Map: 100%                                25000/25000 [00:29<00:00, 971.41 examples/s]
Map: 100%                                25000/25000 [00:27<00:00, 1007.21 examples/s]
Map: 100%                                50000/50000 [01:08<00:00, 816.38 examples/s]

```

```
tokenized_datasets = tokenized_datasets.rename_column("label", "labels")
```

```
tokenized_datasets.set_format("torch", columns=["input_ids", "attention_mask", "labels"])
```

```

train_dataset = tokenized_datasets["train"]
test_dataset = tokenized_datasets["test"]

```

```

model = AutoModelForSequenceClassification.from_pretrained(
    "bert-base-uncased",
    num_labels=2
)

```

```

model.safetensors: 100%                                440M/440M [00:09<00:00, 31.5MB/s]

Loading weights: 100%                                199/199 [00:00<00:00, 618.80it/s, Materializing param=bert.pooler.dense.weight]
BertForSequenceClassification LOAD REPORT from: bert-base-uncased

```

Key	Status
cls.predictions.transform.dense.bias	UNEXPECTED
cls.seq_relationship.bias	UNEXPECTED
cls.predictions.transform.LayerNorm.weight	UNEXPECTED
cls.predictions.transform.LayerNorm.bias	UNEXPECTED
cls.seq_relationship.weight	UNEXPECTED
cls.predictions.bias	UNEXPECTED
cls.predictions.transform.dense.weight	UNEXPECTED
classifier.bias	MISSING
classifier.weight	MISSING

Notes:

- UNEXPECTED :can be ignored when loading from different task/architecture; not ok if you expected
- MISSING :those params were newly initialized because missing from the checkpoint. Consider tra

```

training_args = TrainingArguments(
    output_dir="./results",
    eval_strategy="epoch",
    learning_rate=2e-5,
    per_device_train_batch_size=16,
    per_device_eval_batch_size=16,
    num_train_epochs=2,
    weight_decay=0.01,
    TENSORBOARD_LOGGING_DIR="./logs",
    save_strategy="epoch"
)

```

```

def compute_metrics(eval_pred):
    logits, labels = eval_pred
    predictions = np.argmax(logits, axis=1)

    precision, recall, f1, _ = precision_recall_fscore_support(
        labels, predictions, average='binary'
    )
    acc = accuracy_score(labels, predictions)

    return {
        "accuracy": acc,
        "precision": precision,

```

```

    "recall": recall,
    "f1": f1,
}

```

```

trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=train_dataset,
    eval_dataset=test_dataset,
    compute_metrics=compute_metrics,
)

trainer.train()

```

```

/usr/local/lib/python3.12/dist-packages/torch/utils/data/dataloader.py:775: UserWarning: 'pin_memory' is deprecated. Use 'pin_memory_device' instead.
  super().__init__(loader)

```

 [82/3126 56:40 < 35:56:31, 0.02 it/s, Epoch 0.05/2]

Epoch	Training Loss	Validation Loss
-------	---------------	-----------------

 [107/3126 1:13:44 < 35:20:12, 0.02 it/s, Epoch 0.07/2]

Epoch	Training Loss	Validation Loss
-------	---------------	-----------------

```

results = trainer.evaluate()
print(results)

```

```

predictions = trainer.predict(test_dataset)
y_pred = np.argmax(predictions.predictions, axis=1)
y_true = predictions.label_ids

```

```

cm = confusion_matrix(y_true, y_pred)

plt.figure(figsize=(6,5))
sns.heatmap(cm, annot=True, fmt="d", cmap="Blues")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Confusion Matrix")
plt.show()

print(classification_report(y_true, y_pred))

```